BITS PILANI

Module 05 | AI Platform Engineering

Lesson 8

Distributed Training & Scaling

Data Parallel · Model Parallel · DeepSpeed · FSDP

ADVANCED

Training models too large for a single GPU — on real AWS multi-GPU infrastructure

Learning Objectives

By the end of this lesson, you will be able to:

01

Choose a Parallelism Strategy

Distinguish data, tensor, pipeline, and fully-sharded parallelism — and pick the right one for a given model size and cluster topology.

02

Apply DeepSpeed & FSDP

Use ZeRO optimizer-state sharding (DeepSpeed) and PyTorch FSDP to train models that don't fit on a single device.

03

Scale on AWS

Launch multi-GPU / multi-node training with SageMaker distributed training, understand communication cost, and measure scaling efficiency.

Why One GPU Runs Out

Two independent walls force you to distribute training — long before you 'want' to.

The MEMORY wall

A model's training footprint is far larger than its parameters:

- Weights (FP16): 2 bytes/param
- Gradients: 2 bytes/param
- Optimizer state (Adam): 8+ bytes/param
- Activations: scale with batch × length

A 7B model needs ~80–120 GB to train — more than any single GPU.

The TIME wall

Even if it fit, one GPU is too slow:

- Training data is measured in trillions of tokens
- A single GPU would take months to years
- You need many GPUs working in parallel to finish in days

Distribution solves both — split the memory AND the work.

Data Parallelism: Same Model, Different Data

The simplest and most common strategy — replicate the model, split the batch.

How it works

Every GPU holds a FULL copy of the model. The training batch is split across GPUs — each processes its slice, computes gradients locally, then all GPUs average their gradients (all-reduce) so every replica stays in sync. N GPUs process N× the data per step.

Best for	Models that DO fit on one GPU; you just want to train faster on more data
Limitation	Every GPU stores the whole model — doesn't help if the model itself is too big
Key operation	All-reduce to average gradients each step — communication scales with model size
AWS form	SageMaker Data Parallel (SMDDP), PyTorch DistributedDataParallel (DDP)

When the Model Itself Is Too Big

Data parallelism can't help — you must split the model across devices. Two ways (from Lesson 7).

Tensor Parallelism

Split each layer's weight matrices across GPUs. All GPUs collaborate on the same layer, exchanging partial results.

Intra-layer · heavy communication · needs fast NVLink · low latency.

Pipeline Parallelism

Assign different layers to different GPUs. Data flows through the pipeline stage by stage, like an assembly line.

Inter-layer · light communication · scales across nodes · watch for bubbles.

3D Parallelism

Real large-scale training combines all three: DATA parallelism across groups of GPUs, TENSOR parallelism within a node, and PIPELINE parallelism across nodes. A 175B-parameter model might use 8-way tensor × 8-way pipeline × N-way data parallelism simultaneously. Frameworks like DeepSpeed and Megatron orchestrate this automatically so you configure it rather than build it.

ZeRO: Shard the Redundancy Away

In plain data parallelism every GPU stores identical optimizer state — enormous waste. ZeRO eliminates it.

The core idea

Data parallelism replicates weights, gradients, AND optimizer state on every GPU. For Adam, optimizer state is the biggest chunk (8 bytes/param). ZeRO (Zero Redundancy Optimizer) partitions these across GPUs instead of replicating — each GPU owns a shard, and they gather what they need just in time.

Stage 1	Stage 2	Stage 3
Optimizer State	**+ Gradients**	**+ Parameters**
Partition optimizer state (Adam moments) across GPUs. ~4× memory saving.	Also partition gradients. ~8× memory saving. The common production default.	Also partition the weights themselves. Memory scales with GPU count — train huge models.

DeepSpeed is Microsoft's library implementing ZeRO. ZeRO-Offload/Infinity can even push shards to CPU/NVMe to train billion-parameter models on modest hardware.

FSDP: Fully Sharded Data Parallel

PyTorch's native answer to ZeRO — sharding built into the framework you already use.

1 Shard

Each GPU holds only a shard (slice) of the model's parameters, gradients, and optimizer state — not the whole model.

2 Gather on demand

Just before computing a layer, GPUs all-gather the full parameters for THAT layer only, compute, then free them again.

3 Reshard

After the layer is done, drop the gathered parameters. Peak memory is one layer's worth, not the whole model.

4 Sync

Gradients are reduce-scattered back to their owning shard. Each GPU updates only its slice of the optimizer state.

DeepSpeed ZeRO vs FSDP: same core idea (shard the redundancy). DeepSpeed is more feature-rich (offload, custom kernels); FSDP is native PyTorch, simpler to adopt. Both are first-class on AWS SageMaker.

Communication Is the Bottleneck

Distributed training's real limit isn't compute — it's the network moving gradients between GPUs.

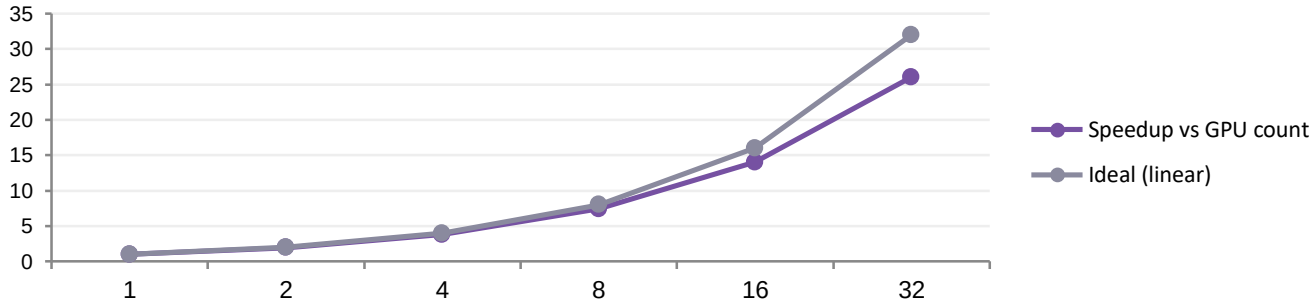
The problem

Every training step, GPUs must synchronise — averaging gradients (all-reduce) or gathering parameters (all-gather). For large models this moves gigabytes between devices every step. If the network is slow relative to compute, GPUs sit idle waiting, and adding more GPUs stops helping.

How AWS addresses it

Purpose-built interconnects: NVLink/NVSwitch between GPUs in a node, and EFA (Elastic Fabric Adapter) between nodes — a high-bandwidth, low-latency network bypassing the OS kernel. P4d/P5 instances pair 8 GPUs with fast NVLink; clusters link them with EFA for near-linear scaling.

Scaling efficiency — the metric that matters



Real speedup falls below ideal as communication overhead grows. 'Scaling efficiency' = actual ÷ ideal. Good distributed setups hold 80–90% even at 32+ GPUs; poor ones collapse. This is what you measure and optimize.

Which Strategy When?

A practical decision path from model size to parallelism choice.

Model fits on 1 GPU, want faster training	**Data Parallel (DDP)**	*Simplest. Replicate + split batch.*
Model fits, but optimizer state is heavy	**ZeRO Stage 1–2 / FSDP**	*Shard optimizer + gradients.*
Model does NOT fit on 1 GPU	**ZeRO Stage 3 / FSDP full shard**	*Shard parameters too.*
Model far exceeds a node's total memory	**Tensor + Pipeline (3D)**	*Split within and across layers.*
Limited GPUs, willing to trade speed	**ZeRO-Offload / Infinity**	*Spill shards to CPU/NVMe.*

Distributed Training on SageMaker

The managed path — SageMaker orchestrates the cluster, networking, and framework integration.

SageMaker Data Parallel

Optimized all-reduce (SMDDP) tuned for AWS network topology — faster than generic NCCL on EFA.

SageMaker Model Parallel

Library that auto-partitions large models with tensor + pipeline parallelism, minimal code changes.

FSDP & DeepSpeed

First-class support — bring your FSDP or DeepSpeed config; SageMaker handles the cluster.

Managed cluster + EFA

P4d/P5 instances with NVLink + Elastic Fabric Adapter provisioned and wired automatically.

You write the training script once; SageMaker provisions the multi-GPU/multi-node cluster, wires the fast network, and runs it. The hands-on lab launches a real distributed job.

Checkpointing & Fault Tolerance

At scale, hardware WILL fail mid-run. Production distributed training plans for it.

Why it matters	A 1,000-GPU job running for days has a high probability of at least one hardware failure. Without checkpoints, a failure at hour 70 means restarting from zero — enormous cost.
Checkpointing	Periodically save model + optimizer state to durable storage (S3). On failure, resume from the last checkpoint instead of the beginning. Sharded checkpoints (each GPU saves its shard) keep this fast.
Elastic / fault-tolerant training	Frameworks (TorchElastic, SageMaker) can detect a failed node, replace it, and resume — keeping the job alive through failures rather than crashing.
The cost trade	Checkpoint too often → waste time on I/O. Too rarely → lose more work on failure. Tune the interval to the job's failure rate and checkpoint cost.

Lab 8: Launch a Distributed Training Job

Run a real multi-worker distributed training job and measure scaling efficiency.

1 Write the training script

~review

A PyTorch script using DDP / FSDP — the same code that scales from 1 to N GPUs.

2 Single-GPU baseline

~3 min

Establish tokens/sec on one device as the reference point for scaling.

3 Launch distributed

~8 min

SageMaker training job across multiple workers with FSDP sharding — real cluster.

4 Measure scaling

~2 min

Compare throughput at 1 vs N workers; compute scaling efficiency (actual ÷ ideal).

5 Checkpoint & resume

~4 min

Save a sharded checkpoint to S3, simulate a failure, resume from it.

6 Report

~2 min

Throughput, scaling efficiency, cost-per-sample across configurations.

Real SageMaker distributed training job · FSDP sharding · scaling-efficiency measurement · S3 checkpointing

Common Pitfalls at Scale

The mistakes that quietly waste GPU-hours — and how to avoid them.

Communication-bound without knowing

Adding GPUs but seeing no speedup — the network is saturated. Fix: profile comms, use gradient accumulation, or a faster interconnect (EFA).

Batch size not scaled

Keeping the single-GPU batch size on N GPUs underutilizes them. Scale global batch with GPU count (and adjust learning rate accordingly).

Ignoring data loading

GPUs starve waiting for data. Fix: parallel data loaders, prefetching, sharded datasets, fast storage (FSx for Lustre).

No checkpointing strategy

One failure at hour 70 restarts from zero. Fix: sharded checkpoints to S3 at a tuned interval.

Wrong parallelism for the model

Using data parallel for a model that doesn't fit. Fix: match strategy to model size (the decision framework).

Not measuring scaling efficiency

Assuming 2× GPUs = 2× speed. Fix: always compute actual ÷ ideal; investigate if below ~80%.

Key Takeaways

Six ideas that define training at scale — and close out Module 05's optimization arc.

1	**Two walls force distribution**	Memory (model too big) and time (data too large). Distribution solves both by splitting each.
2	**Match strategy to the constraint**	Data parallel for speed; shard (ZeRO/FSDP) when memory is tight; model parallel when it truly won't fit.
3	**ZeRO & FSDP shard the redundancy**	Stop replicating optimizer state, gradients, and weights on every GPU — partition them instead.
4	**Communication is the real limit**	Not compute. Fast interconnects (NVLink, EFA) and good overlap are what make scaling efficient.
5	**Plan for failure**	At scale, hardware fails. Sharded checkpointing + elastic training keep long runs alive.
6	**Measure scaling efficiency**	Actual ÷ ideal speedup. Below ~80% means something's wrong — usually communication or data loading.

MODULE 05 OPTIMIZATION ARC · L6 Quantization → L7 Memory & Attention → L8 Distributed Training at Scale